

1. The code implements a recursive factorial function.

The key steps are:

- The main program initializes register x10 to the input value and calls the factorial function 'fact' using the jal instruction.
- The fact function begins by allocating a stack frame using addi to decrement the stack pointer. This reserves space on the stack to preserve register values across recursive calls.
- It then stores the current values of x1 and x10 registers onto the stack using the sd (store doubleword) instruction.
- The input value in x10 is checked to see if it is less than 1, which would be the base case. This is done using bge and addi instructions. If the input is not less than 1, the function makes a recursive call to itself to calculate the factorial of the input minus 1. The jal instruction is used for this recursive call.
- When the recursive call returns, the original x1 and x10 values are restored from the stack using the ld (load doubleword) instruction. The returned factorial value is multiplied with the original input in x10 to calculate the final factorial result.
- Finally, the stack frame is de-allocated using addi and the result is returned to the calling function using jalr.

```

1      addi x10, x10, 6
2      jal fact
3      jal x0, done
4
5 fact:
6      addi sp, sp, -16
7      sd x1, 8(sp)
8      sd x10, 0(sp)
9      addi x5, x10, -1
10     bge x5, x0, L1
11     addi x10, x0, 1
12     addi sp, sp, 16
13     jalr x0, 0(x1)
14 L1:
15     addi x10, x10, -1
16     jal x1, fact
17     addi x6, x10, 0
18     ld x10, 0(sp)
19     ld x1, 8(sp)
20     addi sp, sp, 16
21     mul x10, x10, x6
22     jalr x0, 0(x1)
23 done:

```

Text Segment				
Bkpt	Address	Code	Basic	Source
☐	0x00400000	0x00650513	addi x10,x10,6	1: addi x10, x10, 6
☐	0x00400004	0x008000ef	jal x1,0x00000008	2: jal fact
☐	0x00400008	0x0440006f	jal x0,0x00000044	3: jal x0, done
☐	0x0040000c	0xff010113	addi x2,x2,0xffffffff	6: addi sp,sp,-16
☐	0x00400010	0x00113423	sd x1,8(x2)	7: sd x1,8(sp)
☐	0x00400014	0x00a13023	sd x10,0(x2)	8: sd x10,0(sp)
☐	0x00400018	0xffff5029	addi x5,x10,0xffffffff	9: addi x5,x10,-1
☐	0x0040001c	0x0002d863	bge x5,x0,0x00000010	10: bge x5,x0,L1
☐	0x00400020	0x00100513	addi x10,x0,1	11: addi x10,x0,1
☐	0x00400024	0x01010113	addi x2,x2,16	12: addi sp,sp,16
☐	0x00400028	0x00008067	jalr x0,x1,0	13: jalr x0,0(x1)
☐	0x0040002c	0xffff5051	addi x10,x10,0xffffffff	15: addi x10,x10,-1
☐	0x00400030	0xfdddf0ef	jal x1,0xffffffff	16: jal x1,fact
☐	0x00400034	0x00050313	addi x6,x10,0	17: addi x6,x10,0
☐	0x00400038	0x00013503	ld x10,0(x2)	18: ld x10,0(sp)
☐	0x0040003c	0x00813083	ld x1,8(x2)	19: ld x1,8(sp)
☐	0x00400040	0x01010113	addi x2,x2,16	20: addi sp,sp,16
☐	0x00400044	0x02650533	mul x10,x10,x6	21: mul x10,x10,x6
☐	0x00400048	0x00008067	jalr x0,x1,0	22: jalr x0,0(x1)

Data Segment								
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010140	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010160	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010180	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Registers Floating Point Control and Status		
Name	Number	Value
zero		0 0x0000000000000000
ra	1	0x0000000000400008
sp	2	0x000000007fffffc
gp	3	0x0000000010008000
tp	4	0x0000000000000000
t0	5	0xfffffffffffffffffff
t1	6	0x0000000000000078
t2	7	0x0000000000000000
s0	8	0x0000000000000000
s1	9	0x0000000000000000
a0	10	0x00000000000002d0
a1	11	0x0000000000000000
a2	12	0x0000000000000000
a3	13	0x0000000000000000
a4	14	0x0000000000000000
a5	15	0x0000000000000000
a6	16	0x0000000000000000
a7	17	0x0000000000000000
s2	18	0x0000000000000000
s3	19	0x0000000000000000
s4	20	0x0000000000000000
s5	21	0x0000000000000000
s6	22	0x0000000000000000
s7	23	0x0000000000000000
s8	24	0x0000000000000000
s9	25	0x0000000000000000
s10	26	0x0000000000000000
s11	27	0x0000000000000000
t3	28	0x0000000000000000
t4	29	0x0000000000000000
t5	30	0x0000000000000000
t6	31	0x0000000000000000
pc		0x0000000000400050

2. A. The code implements a recursive computation of Fibonacci numbers.

The key steps are:

- First, the main program initializes register x10 to the input n and calls the fibonacci function fib using the jal instruction which jumps to the function code and saves return address in x1.
- The fib function begins by checking base cases – if n is 0 or 1, it simply returns 0 or 1 respectively. Otherwise, it enters the recursive workflow. It allocates stack frame by decrementing stack pointer x2. This reserves space to save the return address and original input n value.
- It then makes two recursive calls - fib(n-1) and fib(n-2) to compute the terms needed for fib(n). The x1 return address and current n are saved on the stack before the first call. The return value fib(n-1) is stored back on stack before the second call.
- After the two recursive calls return their values, fib(n) is computed by adding the two returned fib values. Finally, the function cleans up the stack frame, loads the return address from stack into x1 and returns to the calling function using jalr instruction.

B. Eventually in x10, 21(decimal) is stored as a hexadecimal number. 21 is written as 0x0000000000000015 in hexadecimal. It is highlighted in the picture below.

```
1  .data
2
3  .text
4      _main:
5          addi x10,x10,8
6          jal fib
7
8          li a7,1
9          ecall
10
11         li a7,10
12         ecall
13
14         addi x10,x10,8
15 fib:|
16         beq x10, x0, done
17         addi x5, x0, 1
18         beq x10, x5, done
19         addi x2, x2, -16
20         sd x1, 0(x2)
21         sd x10, 8(x2)
22         addi x10, x10, -1
23         jal x1, fib
24         ld x5, 8(x2)
25         sd x10, 8(x2)
26         addi x10, x5, -2
27         jal x1, fib
28         ld x5, 8(x2)
29         add x10, x10, x5
30 # Clean up:
31         ld x1, 0(x2)
32         addi x2, x2, 16
33 done:
34         jalr x0, 0(x1)
35
```

21

-- program is finished running (0) --

Clear

Text Segment					
Bkpt	Address	Code	Basic	Source	
	0x00400000	0x00850513	addi x10,x10,8	5:	addi x10,x10,8
	0x00400004	0x018000ef	jal x1,0x00000018	6:	jal fib
	0x00400008	0x00100893	addi x17,x0,1	8:	li a7,1
	0x0040000c	0x00000073	ecall	9:	ecall
	0x00400010	0x00a00893	addi x17,x0,10	11:	li a7,10
	0x00400014	0x00000073	ecall	12:	ecall
	0x00400018	0x00850513	addi x10,x10,8	14:	addi x10,x10,8 # ...
	0x0040001c	0x04050063	beq x10,x0,0x00000040	16:	beq x10, x0, done # ...
	0x00400020	0x00100293	addi x5,x0,1	17:	addi x5, x0, 1
	0x00400024	0x0250c63	beq x10,x5,0x00000038	18:	beq x10, x5, done # ...

Data Segment								
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

0x10010000 (.data) ☒ Hexadecimal Addresses ☒ Hexadecimal Values ☐ ASCII

Registers Floating Point Control and Status		
Name	Number	Value
zero	0	0x0000000000000000
ra	1	0x0000000000400008
sp	2	0x000000007ffffeffc
gp	3	0x0000000010008000
tp	4	0x0000000000000000
t0	5	0x000000000000000d
t1	6	0x0000000000000000
t2	7	0x0000000000000000
s0	8	0x0000000000000000
s1	9	0x0000000000000000
a0	10	0x0000000000000015
a1	11	0x0000000000000000
a2	12	0x0000000000000000
a3	13	0x0000000000000000
a4	14	0x0000000000000000
a5	15	0x0000000000000000
a6	16	0x0000000000000000
a7	17	0x000000000000000a
s2	18	0x0000000000000000
s3	19	0x0000000000000000
s4	20	0x0000000000000000
s5	21	0x0000000000000000
s6	22	0x0000000000000000
s7	23	0x0000000000000000
s8	24	0x0000000000000000
s9	25	0x0000000000000000
s10	26	0x0000000000000000
s11	27	0x0000000000000000
t3	28	0x0000000000000000
t4	29	0x0000000000000000
t5	30	0x0000000000000000
t6	31	0x0000000000000000
pc		0x0000000000400018

3. A. The stack pointer allows a function to save and restore the registers used by it, so as not to modify registers needed by the calling function. Function arguments can be passed on the stack by pushing them before the call instruction and having the called function access them using the stack pointer.
- B. Basic Block is a sequence of instructions with no embedded branches (except at end) and has no branch targets (except in the start).
- C. The NOT logical operation can be implemented in RISC-V using the xori (exclusive OR immediate) instruction.
- D. When we use the sb instructions it stores the least significant byte of a register to memory. When we use the lb instruction, it loads a single byte from memory into a register. lb instruction requires sign extension.